

TEAM DETAILS AND GITHUB REPOSITORY LINK:

GitHub repository link: <https://github.com/reneilwemolopyane/COS214-Prac6.git>

Team Details

STUDENT NAME AND SURNAME	STUDENT NUMBER
Lizalise Mbonisweni	u23587874
Boikemelo Masoka	u25128648
Thembelisha Skosana	u25224663
Lethabo Molobi	u25090209
Boitumelo Monareng	u25208943
Reneilwe Molopyane	u25161874
Dingalethu Ngumbela	U25170547

Workflow-management research findings and references

Theme 1: workflow definitions and running workflow instances

A Workflow Management System is a system that defines, manages, and executes workflows using software based on the workflow logic. It automates the execution of applications and is robust against performance variations and failures .A workflow definition is a structured blueprint or electronic template that outlines the exact sequence of steps, rules, and participants required to complete a specific task or business process .Each time a workflow definition is published, a new version is created. By default, the new version is used by existing instances of the workflow. If a stage is removed, active instances on the stage continue to use the previous version until they progress to a stage that is available in the newly published version .A 'Workflow Instance' is a specific occurrence of a workflow that is currently running or being executed, involving a series of interconnected computational tasks with data and control dependencies.

Reference:

ScienceDirect, <https://www.sciencedirect.com/topics/computer-science/workflow-management-system>

By Kaya Souaïbou Hawaou, ... Laurent Bitjoka

IBM, <https://www.ibm.com/docs/en/openpages/9.0.0?topic=fundamentals-workflow-definitions-workflow-instances>

By IBM

Theme 2: work items, stages and sub-processes

A workflow can be structured at different levels, with individual work items grouped into larger stages or sub-processes to represent complex processes more clearly. Work items can also have dependencies, meaning one task may need to be completed before another can begin, while some tasks can run in parallel and later join together. During execution, work items can move through different states such as available, assigned, running, completed or suspended, so TIVIDY needs to track their current state, completed work and what can happen next.

Theme 3 : Assignment and routing

Resources are modelled separately from the process, and a resource is anything capable of doing work, whether a person or a piece of equipment. People sit in an organisational structure of positions, units and teams, and have roles, capabilities, schedules and histories (Russell et al., 2004). Roles separate who does the work from the process definition. A task names a role such as Manager, and the person who fills it is only chosen when the work item becomes runnable, so one definition can serve many instances with different people (Russell et al., 2004). Allocation strategies are also kept separate from the work itself. Some are fixed at design time, such as role-based, capability-based, history-based and separation of duties, where the person who prepares a check cannot countersign it. Others select among eligible people, for example at random, round robin or shortest queue, and allocation can happen early, when the item becomes enabled, or late (Russell et al., 2004). A system can offer an item to one resource or to many, where the first to claim it wins, or allocate it directly. Under push the system assigns the item, and under pull the resource claims it. A work item moves from created to offered, allocated, started and completed, with suspended and failed as exceptions, and each change is started either by the system or by the resource. Assignment can also change through delegation, deallocation and escalation, where a stalled item is automatically reassigned after a deadline passes, and some tasks are automatic and need no person at all (Russell et al., 2004). For TIVIDY, this means roles, allocation rules and escalation deadlines belong to the workflow definition, so they can change without rewriting the work-item classes. The specific participants assigned, their workload and the assignment history belong to each running instance.

Sources

- **Russell, N., ter Hofstede, A. H., Edmond, D., & van der Aalst, W. M. (2004).** *Workflow Resource Patterns*. Technische Universiteit Eindhoven.
Link: <https://pure.tue.nl/ws/files/1984893/591788.pdf>

Theme 4: Approvals, Rejection, Escalation

Approval is a normal workflow state and not an error: the workflow pauses until a decision arrives, then follows the matching outcome of approval, rejection or escalation (Temporal Technologies, 2026). Approval rules vary, since steps can be sequential or parallel and may need every approver, a quorum or any one approver, while a single rejection can veto the step (django-workflow-kit, n.d.). Rejection stops the request from reaching later steps, and the approver records a reason that stays on the request (ServiceNow, n.d.). Escalation prevents work from stalling: if no decision arrives in time, the workflow re-notifies the approver, then their manager, then applies a default decision, and approvers can delegate when unavailable (EmpowerID, n.d.). For TIVIDY, approval modes, deadlines and escalation chains belong to the workflow definition, while approval status, decision reasons and escalations triggered belong to each running instance. This suggests Chain of Responsibility for the approval chain and Decorator for adding deadlines, notifications and auditing to a work item.

Sources:

Temporal Technologies, 2026. Approval pattern. Temporal Documentation.

<https://docs.temporal.io/design-patterns/approval>

django-workflow-kit, n.d. Approvals. django-workflow-kit Documentation.

<https://django-workflow-kit.readthedocs.io/en/stable/approvals/>

EmpowerID, n.d. Escalations. Workflow Studio Developer Guide.

<https://docs.empowerid.com/docs/developer-guide/workflow-studio/concepts/escalations>

ServiceNow, n.d. Approving approval requests. ServiceNow Documentation.

<https://www.servicenow.com/docs/r/order-management/approving-approval-requests.html>

Theme 5: Events and Failures

A workflow system must react to events such as a task finishing, a deadline passing, a message arriving or an error occurring. Russell, van der Aalst and ter Hofstede (2006) identify five types of exception: work item failure, deadline expiry, resource unavailability, external triggers and constraint violations. Each must be handled at the work-item level, at the whole-case level and with a recovery action such as rollback or compensation. Planning matters, because unexpected exceptions slow processes down significantly more than expected ones (Dijkman et al., 2019). Modern engines therefore build failure handling into the workflow definition: temporary failures are retried automatically, business errors are sent down an alternative path, completed work can be undone through compensation, and failures that still can't be resolved

raise an incident for a person to fix (Camunda, 2025; Temporal Technologies, 2026). Handlers can even be chosen at runtime based on the situation (Adams et al., 2007). For TIVIDY, this means failure rules belong to the workflow definition, while retry attempts, actual errors and open incidents belong to each running instance.

Theme 6: Integration & Reusability

Workflow systems keep the engine separate from external systems through defined interfaces: the WfMC Reference Model has dedicated interfaces for invoking external applications and for communicating with other engines (Workflow Management Coalition, 1995), and Camunda uses outbound connectors to call external systems and inbound connectors to bring events in (Camunda, 2025). Integration details are configuration, not code: Power Automate's connection references let one connection be reused across flows and environments without editing each action (Microsoft, 2024). For TIVIDY, this means a generic integration abstraction (databases, APIs, notifications) with configurable endpoints, and routing and assignment rules supplied per workflow, so our scenario is one application of a reusable engine.

Sources

Workflow Management Coalition (1998). *The Workflow Reference Model*.

<http://www.workflowpatterns.com/documentation/documents/tc003v11.pdf>

Camunda. *Glossary*, Camunda 8 Docs. <https://docs.camunda.io/docs/reference/glossary>

Microsoft. *Use connector actions in desktop flows*, Microsoft Learn. <https://learn.microsoft.com/en-us/power-automate/desktop-flows/how-to/use-connector-actions>

Theme 7(history,versioning,monitoring)

What is audit information?

Audit information is a standardized collection of data recording system activities and participant actions during process execution. Based on WfMC Interface 5, it serves as an audit trail that documents specific workflow events, participant interactions, and instance execution data to track process history.

What is a workflow history?

chronological log of runtime events that records the full execution path of a specific workflow instance. It tracks state transitions across task lifecycles (such as tasks being

scheduled, assigned, started, completed, failed, or cancelled), serving as the foundational log for durable execution, crash recovery, state replay, and debugging

What is workflow versioning?

The operational mechanism that manages updates to process definitions over time without disrupting ongoing operations. It enforces a strict boundary between process blueprints and active executions, ensuring that running workflow instances remain stable even as new process models are deployed

Importance:

Managing Real-World Process Evolution

- **Adapting to Process Changes:** Workflows represent evolving real-world processes (laboratory procedures, business approvals, manufacturing steps). Versioning allows systems to update process definitions and underlying code to reflect new operational needs without rewriting the entire framework from scratch.

Supporting Multiple Concurrent Versions

- **Simultaneous Execution:** In many systems, existing materials or tasks cannot simply be migrated or discarded when a workflow definition changes. Versioning enables a workflow engine to run multiple process versions simultaneously, allowing active instances to complete using their original definition while routing new instances through the updated process.

Ensuring Code and Execution Integrity

- **Preventing Inconsistencies:** Updating process logic or step code without version tracking can break running instances or lead to unexpected behavior. Versioning ensures that the state and history of each material remain consistent with the exact code and schema version under which it was initiated.

Tracking History and Auditability

- **Historical Accuracy:** A material's state is determined by its history of completed steps. Versioning preserves the precise definition of past steps and logic, ensuring accurate record-keeping, debugging, and auditability over time.

What is monitoring

Process monitoring deals with the analysis and representation of process instances at run time. Using monitoring information, workflow administrators and process managers can manipulate the behavior of current workflow instances and react to problems that

arise during process enactment. Furthermore, process monitoring is used to improve the responsiveness of an organization to customer inquiries.

Importance :

In workflow management and process controlling, **process monitoring** plays a critical operational role.

- **Real-Time Operational Visibility:** It provides workflow administrators and process managers with immediate, run-time insight into active process instances, current execution states, and live performance metrics.
- **Proactive Problem Resolution:** By highlighting bottlenecks, delays, or errors as they happen, monitoring allows managers to intervene and adjust current workflow instances to resolve operational issues dynamically.
- **Enhanced Customer Responsiveness:** It enables organizations to handle customer inquiries quickly and accurately. When the exact state and handler of a process instance can be identified instantly, questions such as "Who is processing order #4711?" can be answered without delay.
- **Foundation for Cybernetic Control:** Monitoring acts as the sensor in a cybernetic feedback loop, feeding accurate runtime data back into the system to keep operational performance aligned with target goals and standard operating procedures.

How workflow History and Audit Information are Recorded?

Workflow Management Systems (WFMSs) log execution events, including activity start and completion timestamps, input/output data, assigned resources, and execution outcomes.

How it is extracted:

Pull Strategy: Reading log files or database tables generated by the WFMS.

Push Strategy: Receiving real-time event notifications via messaging software/middleware.

Raw audit events pass through an ETL Container where they are parsed, standardized, and correlated across three levels of execution (workitems, activity instances, and process instances). The transformed history is then loaded into fact and dimension

tables within a Process Data Store (for operational/near real-time monitoring) and a Process Warehouse (for strategic analysis).

Separation of Runtime State and History Event Streams

Workflow management systems maintain a strict physical separation between their **runtime database** (which manages active process states and immediate task routing) and their **history event stream** (which stores immutable audit logs). To prevent database performance degradation and server crashes under high volume, engines discriminate between lightweight **process data** (gateway decisions, routing variables, business keys) stored in the engine and heavy **application payloads** (such as invoices or clinical data) stored in dedicated external databases

Standardized Audit Event Schemas (CWAD Framework)

To allow consistent auditing across heterogeneous workflow engines, audit events follow the WfMC Interface 5 standard, structuring every record into a prefix and suffix format:

- **Mandatory Prefix Information:** Header attributes including InitialProcessInstanceID (the root process key), CurrentProcessInstanceID, UserID, RoleID, DomainID, NodeID, Timestamp, and standard EventCode values (such as WMCreatedProcessInstance, WMCompletedActivityInstance, or WMChangedWorkItemState).
- **Suffix Extensions:** Optional extension blocks that carry process-relevant business variables, account codes, and custom metadata.

Because workflow systems often run across distributed, heterogeneous environments, history and audit records are structured for transparency and persistence:

- **Distributed CORBA Objects:** Each process or task instance maintains its history as an independent CORBA object located at the same site where the activity is executed to minimize network communication overhead.
- **Data Repositories (DBMS & Files):** Storage wrappers map history objects to persistent storage engines:
 - **Relational Database Mapping:** When a DBMS is available, history is mapped into relational tables:
 - **PROCESS table:** Stores process metadata.

- TASK table: Stores task execution details.
 - PROCESS_CHILD table: Tracks parent-child relationships between processes and their associated sub-tasks or subprocesses.
 - ACTIVITY_DATA table: Captures input and output parameter states across activity versions.
- **File Repositories:** On sites lacking a DBMS, history objects are persisted directly as structured files supporting relational-style query operations.

Sources:

Zur Muehlen, M., 2004. *Workflow-based process controlling: foundation, design, and application of workflow-driven process information systems* (Vol. 6). Michael zur Muehlen.

Schiefer, J., Jeng, J.J. and Bruckner, R.M., 2003, September. Real-time workflow audit data integration into data warehouse systems. In *ECIS* (pp. 1697-1706).

Koksal, P., Arpinar, S.N. and Dogac, A., 1998. Workflow history management. *ACM Sigmod Record*, 27(1), pp.67-75.

Rolfe, P.A., 2001. *Code versioning in a workflow management system* (Doctoral dissertation, Massachusetts Institute of Technology).

<https://docs.camunda.io/docs/components/best-practices/operations/versioning-process-definitions/>

TASK2 TIVIDY is a generic workflow management system. It stores descriptions of processes, creates independent running copies of them, coordinates the work inside each copy, and keeps a record of everything that happens. It is not tied to any particular organisation or industry.

Main responsibilities:

- **Define workflows:** store workflow definitions made of stages, work items, sub-processes, dependencies and rules, and keep older versions of definitions.
- **Create and run instances:** start independent workflow instances from a definition, each with its own data, participants, progress and history.
- **Manage work items:** track each work item through its lifecycle (created, available, assigned, started, completed, rejected, cancelled, escalated) and only allow operations that are valid in the current state.
- **Route and assign work:** decide what becomes available next, and who receives each work item, using rules that can be changed.
- **Handle approvals and escalation:** support single and multi-level approvals, rejections and deadlines, and escalate when no one responds.
- **React to events:** respond to internal events (a work item completes) and external ones (a document arrives, a deadline passes), and notify any interested parts of the system.
- **Communicate with external systems:** exchange information with outside systems through a controlled boundary.
- **Record history:** retain actions and state changes so the system can explain what happened, and support undo or rollback where it makes sense.
- **Monitor and report:** provide views of progress, delays and outcomes without placing every report inside the core model.
-

Boundaries (What TIVIDY does not do)

- It coordinates work but does not perform the work itself. People and external systems carry out tasks, and TIVIDY tracks and routes them.
- It does not contain the business rules of any one organisation. Rules are supplied through configurable definitions.
- It does not own external systems. It communicates with them through connectors that translate between TIVIDY and their interfaces.
- It does not decide what a process should be. It runs what the definition describes.

Definition vs running instance

Belongs to the workflow definition	Belongs to a running instance
Stages, work items and sub-processes	Current stage and state of each work item
Dependencies and ordering rules	Which dependencies have been satisfied
Roles allowed to perform each item	The specific participants assigned
Routing, assignment and approval rules	Decisions actually made along the way
Deadlines and escalation rules	Actual start times, due times, escalations triggered
Version number	The definition version it was started from
Data requirements (what must be collected)	The actual data values and attachments
	History of actions and state changes

Changing one instance must never change the definition or any other instance

Parts expected to vary

- How work is assigned (by role, workload, or a rule calculated from data)
- How approvals work (one person, several levels, escalation)
- The lifecycle behaviour of work items, and extra behaviour layered on (validation, priority, auditing, notifications, security)
- The structure of work (sequential, parallel, nested)
- Notifications and event reactions, since new interested parties should be addable
- External integrations, since new systems should be connectable without redesigning the engine
- Reports and views of running workflows
- How workflows are built (different ways of constructing definitions)

Design rationale

Process descriptions are kept separate from running processes so that many instances can run from one definition without interfering. The parts listed as varying are isolated behind stable abstractions, so a new rule, integration or report can be added without rewriting the engine or the work-item classes. The chosen scenario will run on TIVIDY; it will not define TIVIDY.

Selected Scenario Description

1. Organisation

- A short-term insurance company that handles motor vehicle claims, with a call centre, claims department, assessors, a fraud unit and approved panel beaters.

2. Process being managed

- A motor vehicle accident claim, from the moment a policyholder reports an accident until the car is repaired and paid for, or the claim is rejected.

3. Participants and roles

- **Policyholder:** reports the claim and uploads documents
- **Claims handler:** checks the claim and manages it end to end
- **Assessor:** inspects the damaged car and estimates the cost
- **Panel beater:** quotes for and repairs the car
- **Claims manager / senior manager:** approve claims above certain amounts
- **Fraud investigator:** reviews suspicious claims
- **Finance clerk:** releases payment

4. Major pieces of work

- Lodge claim → verify policy → collect documents (police case number, photos, driver's licence, quotes) → assess damage → decide → authorise repair → repair → final inspection → pay → close

5. Decisions and routing points

- Is the policy active and does it cover this type of accident?
- Is the car repairable or a **write-off**? A write-off leads to a cash payout instead of a repair.
- Does the claim score as **suspicious**? If so, it is routed to the fraud investigator.
- How much is it? The amount decides which approval level is needed.

6. Approvals and dependencies

- Approval levels by amount, e.g. handler up to R20 000, manager up to R100 000, senior manager above that.
- **Parallel work:** the police report check, assessor inspection and repair quotes happen at the same time, and the decision waits for all three.
- Repair can't start until approval is given; payment can't happen until the final inspection passes.

7. Failures and exceptions

- Policy lapsed or excess not paid: claim rejected.
- Documents missing: request sent to the policyholder; escalate if there's no response in 7 days.
- Approver doesn't respond in time: **escalate** to the next level.
- Fraud confirmed: claim rejected, and any payments already made are reversed (**compensation**).
- Payment gateway or police system down: **retry**, then raise an **incident** for a person to fix.
- Repair fails the final inspection: send back to the panel beater (**loop**).

8. External systems

- Policy administration system (check cover)
- Police case verification, using the case number
- Vehicle valuation service (write-off decisions)
- Panel beater portal (quotes and repair status)
- Payment gateway / bank
- SMS and email notifications

9. Expected outcome

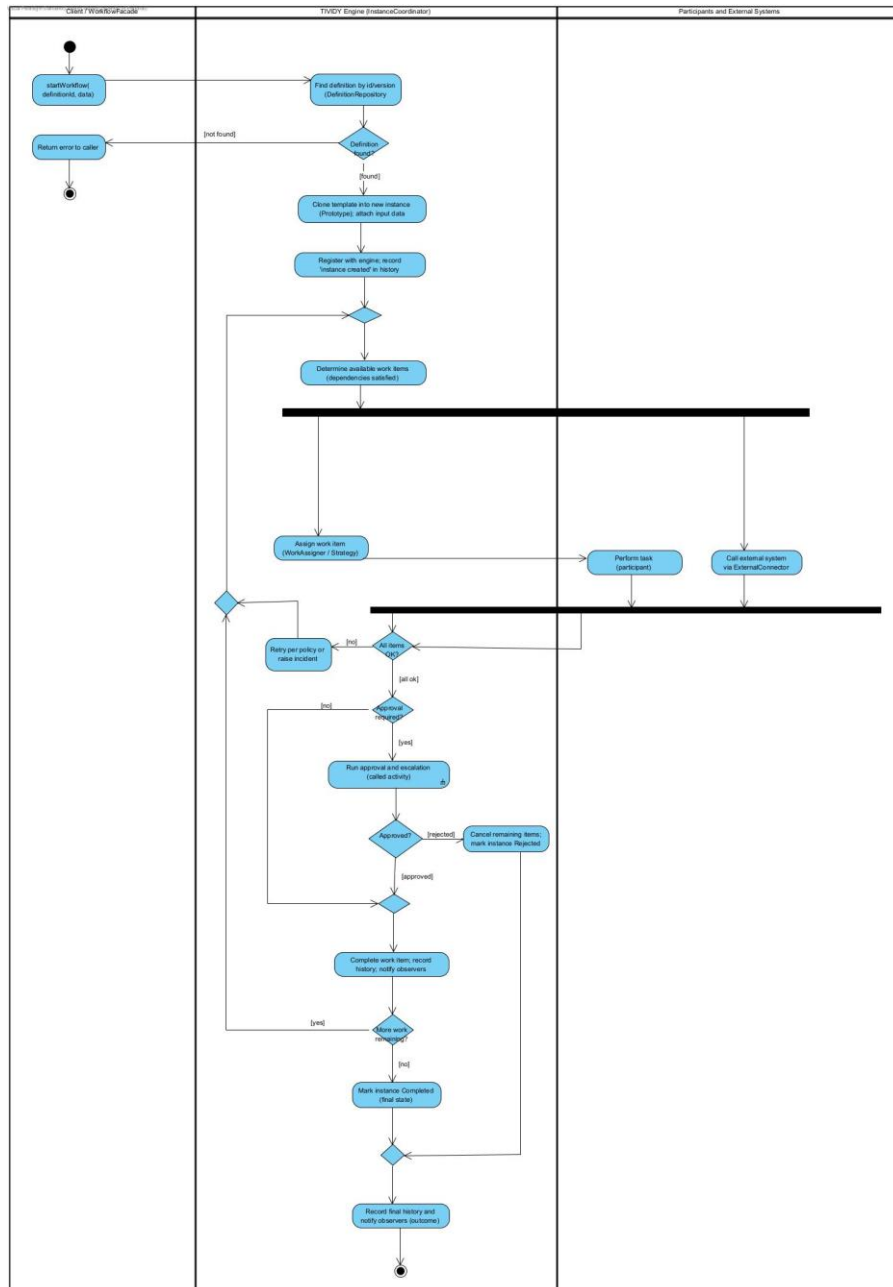
- **Approved:** car repaired, or write-off paid out; policyholder notified; claim closed.
- **Rejected:** policyholder notified with the reason; claim closed.
- In both cases, a full **audit history** of every decision and action is kept.

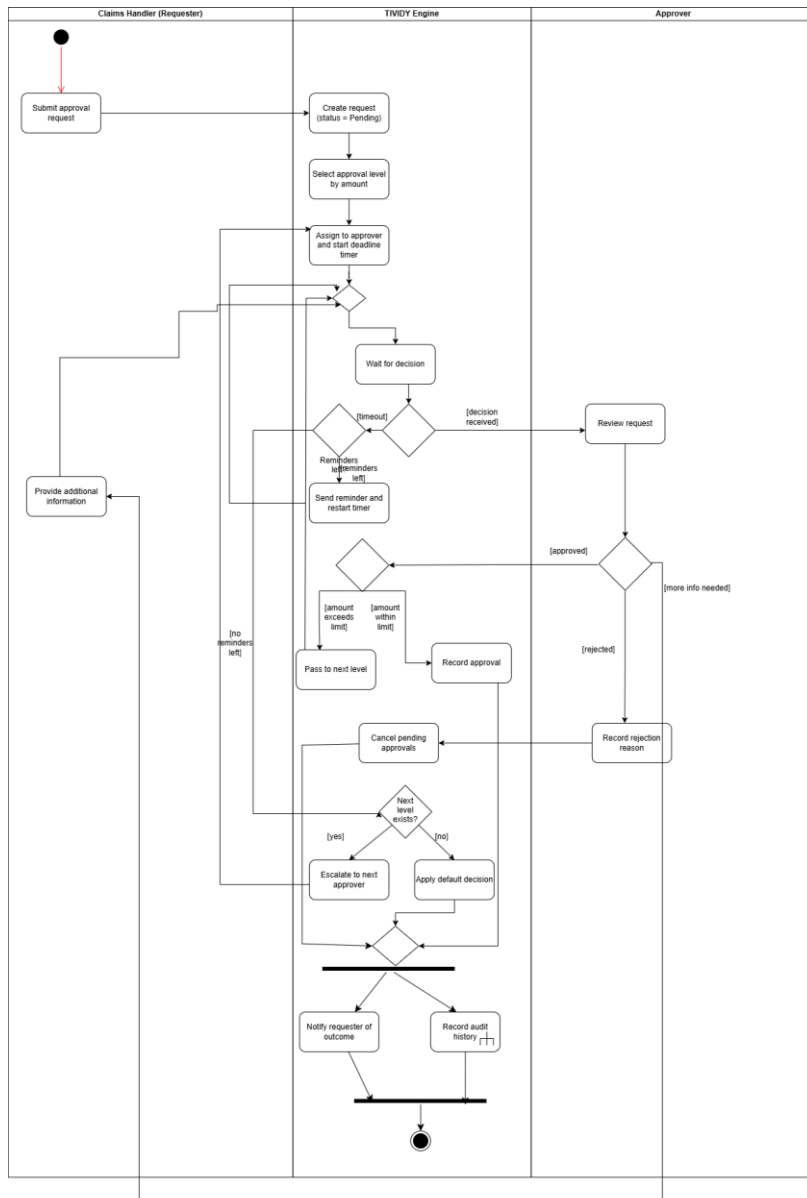
Task 3 - The three UML Activity Diagrams

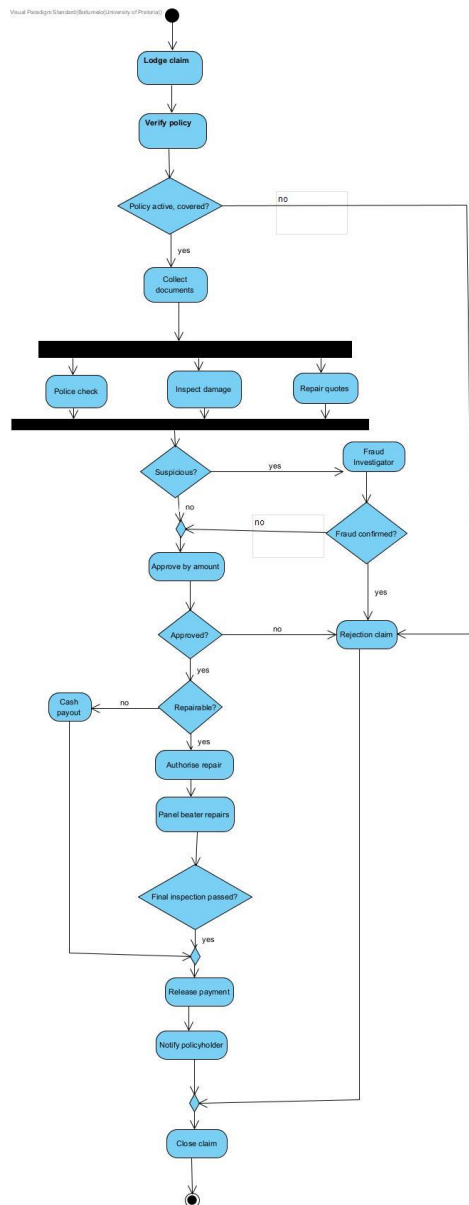
1. A generic workflow execution showing the progression of a workflow instance from creation or start until completion or another appropriate final outcome.

Progress of a workflow instance from startWorkflow() to Completed or Rejected.

Generic because there is no insurance logic.







Task 4 – Design Patterns

GoF Participants & Justifications

1. Strategy

- Strategy: AssignmentStrategy
- ConcreteStrategy: RoleBasedStrategy, ShortestQueueStrategy, RoundRobinStrategy, DataRuleStrategy
- Context: WorkAssigner
- **Justification:** Different organisations assign and route work differently: by role, by workload, or by a rule calculated from data. Each way of choosing a participant becomes its own class behind the AssignmentStrategy interface.

2. Mediator

- Mediator: WorkflowMediator
- ConcreteMediator: Instance Coordinator
- Colleague: WorkflowComponent
- ConcreteColleague: WorkItem, WorkAssigner, ApprovalHandler and ExternalConnector
- **Justification:** We need Mediator because the parts of a running workflow have to coordinate with each other, and without it they would all depend on each other directly.
- In TIVIDY, finishing one work item triggers several things: checking whether the next item's dependencies are met, making it available, assigning it to a person, and notifying them. In the insurance scenario the police check, assessor inspection and repair quotes run in parallel, and the decision must wait for all three. If work items, the assigner, the approvers and the external connectors each called one another, every class would need to know about all the others. Adding a new part would mean changing many classes.
- With a Mediator, each part only talks to InstanceCoordinator by calling `notify(sender, event)`, and the coordinator decides what happens next. The dependency checks and the order of reactions live in one place, so a rule can change without editing the work-item classes. There is one coordinator per workflow instance, which fits your definition-versus-instance split: it holds instance data such as which dependencies are satisfied, so instances never interfere with each other.
-

3. Builder

3.1 Design problem: A workflow definition is a large structure of stages, work items, dependencies and rules that must be assembled in a valid order. Organisations will supply definitions in different ways and the definition class shouldn't be rewritten for each.

3.2 Participants:

- Builder – WorkflowBuilder
- ConcreteBuilder1 – ValidatingWorkflowBuilder
- ConcreteBuilder2 – DraftWorkflowBuilder
- Director – WorkflowDirector
- Product – WorkflowDefinition

3.3 Collaboration: The Director takes a builder and calls the build steps (buildStage, buildWorkItem, buildDependency, buildRule) in a valid order. The caller then asks the builder for the finished WorkflowDefinition with getResult().

3.4 What varies: How a definition is assembled and where its description comes from.

3.5 Why it fits: Construction logic is separated from the definition, so adding a new source of definitions means adding a ConcreteBuilder, not changing the engine or WorkflowDefinition.

4. Prototype

4.1 Design problem: Many instances must be started from one definition, and each needs its own progress, data and history. Changing one instance must never change the definition or another instance.

4.2 Participants:

- Prototype – WorkComponent
- ConcretePrototype1 – WorkItem
- ConcretePrototype2 – Stage
- ConcretePrototype3 – SubProcess
- Client – WorkflowDefinition

4.3 Collaboration: createInstance(data) calls clone() on the root template. Composite nodes clone their children recursively, so the instance receives a fully separate copy with fresh state, which WorkflowDefinition wraps in a new WorkflowInstance.

4.4 What varies: The kinds of work node. New types only need their own clone()

4.5 Why it fits: Instances come from copying a template structure, not from rebuilding it each time, and the deep copy is what guarantees independence between instances.

5. Facade

5.1 Design problem: Starting and querying a workflow involves several subsystems (repository, definition, instance, engine). Callers like a UI or an external trigger shouldn't need to know the order or details.

5.2 Participants:

- Facade – WorkflowFacade
- Subsystem class – DefinitionRepository
- Subsystem class – WorkflowDefinition
- Subsystem class – WorkflowInstance
- Subsystem class – WorkflowEngine

5.3 Collaboration: The client calls `startWorkflow(definitionId, data)`. The facade finds the definition, asks it to create an instance, registers the instance with the engine and starts it, then returns the instance id.

5.4 What varies: The internals of storage and the engine.

5.5 Why it fits: It gives TIVIDY one stable entry point, so the internals can be reorganised without breaking callers.

6. Adapter

6.1 Design problem: TIVIDY must communicate with multiple external systems whose interfaces are incompatible and cannot be changed.

6.2 Participants:

- Prototype – WorkComponent
- ConcretePrototype1 – WorkItem
- ConcretePrototype2 – Stage
- ConcretePrototype3 – SubProcess
- Client – WorkFlowDefinition

6.3 Collaboration: When a work item requires external data or an external operation, InstanceCoordinator uses ExternalConnector to issue a generic request. The appropriate Adapter translates this request into the interface expected by its Adaptee and converts the external response into a common

ConnectorResult. The coordinator then decides whether the workflow continues, retries, or escalates.

6.4 What varies: The external system, its API and integration-specific configuration vary between integrations. Adding another external system therefore requires a new Adapter rather than changes to the workflow engine or work-item classes.

6.5 Why it fits: Adapter gives TIVIDY a consistent interface for external integrations while isolating incompatible external APIs from the core workflow model. This supports the practical's requirement for a generic and reusable workflow management system.

Memento

Design Problem: The Memento pattern captures and externalises the internal execution state of a running WorkflowInstance without exposing private attributes or breaking object encapsulation. It physically decouples volatile runtime state from immutable history event streams, enabling lightweight task routing while preserving compliance records. This provides a clean mechanism for safe crash recovery, event-sourced replay, and state rollbacks to historical checkpoints without corrupting active engine data.

Participants

Originator-InsuranceClaimInstance

Memento- ClaimInstanceMemento

Caretaker-ProcessDataStore

Collaboration:

- **Snapshot Creation:** Whenever an activity state changes or a milestone occurs, Caretaker requests a state snapshot from Originator.
- **Encapsulated State Capture:** InsuranceClaimInstance instantiates a new ClaimInstanceMemento, populating it with a deep copy of its current internal variables and execution state.

Immutable Persistence: The InsuranceClaimInstance returns the memento to ProcessDataStore, which appends it to the immutable history event log.

- **State Restoration:** If a system crash or failure occurs, ProcessDataStore retrieves the appropriate ClaimInstanceMemento and passes it back to InsuranceClaimInstance e.g. `restoreFromMemento()`, resetting the instance to its exact historical state

What varies:

The specific state data (business variables, stage parameters, payload map) recorded inside a memento varies per workflow definition and scenario.

Why it fits:

The Memento pattern directly satisfies the core requirement of maintaining an immutable workflow execution history while preserving object encapsulation. Without Memento, the ProcessDataStore would require direct access to private internal attributes of

InsuranceClaimInstance ,creating tight coupling and exposing sensitive runtime state to accidental external mutation.

Visitor

Design problem: Hardcoding reports and monitoring into workflow tasks bloats the code and forces you to edit task classes every time you want a new report.

The Visitor pattern moves monitoring into separate classes using `accept()` . This lets you add new analytics and reports without touching or breaking core workflow execution code.

Participants:

Abstract Element Interface: WorkflowElement

Concrete Elements: WorkflowInstance

Abstract Visitor Interface: WorkflowVisitor

Concrete Visitors:

- BottleneckVisitor
- SLAComplianceVisitor
- AuditReportVisitor

Collaboration

- Invocation: The client invokes `accept(visitor)` on a root element (e.g., WorkflowInstance), passing a concrete visitor like BottleneckVisitor.
- Double Dispatch Handshake: Inside `WorkflowInstance::accept(visitor)`, the instance calls `visitor.visitInstance(this)`, granting the visitor read access to its attributes.
- Composite Traversal Propagation: Container elements (WorkflowInstance and Stage) propagate the `accept(visitor)` call to all child elements in the process hierarchy.
- Data Gathering: As the visitor visits each WorkItem, it extracts necessary operational metrics and aggregates them internally to produce monitoring reports without altering task execution logic.

What Varies:

Monitoring & Reporting Algorithms: The types of operational analysis, metric calculations, and compliance report formats can vary or expand indefinitely without modifying any core workflow execution classes.

Why it fits:

The Visitor pattern enforces the Open/Closed Principle for process monitoring and telemetry. Core task execution classes remain lightweight and focused solely on process routing, while developers can add arbitrary new monitoring dashboards, SLA checkers, or compliance exporters simply by creating new WorkflowVisitor subclasses.

7 OBSERVER

7.1 The design problem in TIVIDY that the pattern solves

When a work item in a running instance completes, fails or is escalated, several independent parts of TIVIDY must react: the dependency tracker, the audit logger, the notification service

and the incident monitor. Observer lets the WorkflowInstance announce each change once to whichever listeners have subscribed, so it never depends on them directly and new reactions can be added without modifying it.

7.2 Participants

GoF role	TIVIDY Class	Responsibility
Subject	WorkflowSubject (abstract)	Keeps observers per event type attach(). detach(), notify()
ConcreteSubject	WorkflowInstance	Stores its current state as a WorkflowEvent; setState() updates it and calls notify; getState returns it
Observer	WorkflowObserver (abstract)	Keeps a reference to the subject it observes; declared update().
ConcreteObserver	DependencyTracker	On COMPLETED, releases work items whose predecessors are all done.
ConcreteObserver	AuditLogger	Records every event in the instance's audit trail
ConcreteObserver	NotificationService	Notifies the participant who must act next (via an ExternalService, e.g. SMS/email).
ConcreteObserver	IncidentMonitor	On FAILED, asks the instance's CommandProcessor to retry failed command; otherwise raises an incident.
Event data	WorkflowEvent, EventType	Carries the type, instance, work item, time, details and the command that caused it (pull model)

7.3 Collaboration

1. Each concrete observer is attached to a WorkflowInstance through attach(), and keeps a reference to that instance as its subject.
2. When something changes, the instance's setState(e) stores the new WorkflowEvent and calls notify().
3. notify() calls update() on every observer in the observerList.
4. Each observer **pulls** the state by calling subject->getState(), then reacts to the event type it cares about.
5. When an observer is destroyed, its destructor calls detach(), so the subject never holds a dangling pointer.

7.4 What part of the design is expected to vary

The part of the design expected to vary is the set of observers that react to workflow events. New reactions, such as a deadline monitor or a progress dashboard, can be added by subclassing WorkflowObserver and attaching them, without changing WorkflowInstance.

7.5 Why is Observer appropriate

This is a one-to-many dependency in which the object that changes must not know who depends on it. Calling every reaction directly would tightly couple the instance to logging, notification and failure handling, while polling the instance for changes would waste work and delay reactions. Observer gives a separation of concerns: observers are independent of the instance and can register or deregister as needed. TIVIDY uses the pull model, so the subject does not need to know what data each observer requires. Each observer fetches the event through getState().

8 COMMAND

The design problem in TIVIDY that the pattern solves

Actions on a workflow, such as assigning or completing a work item or calling an external service like a payment gateway, must be recorded for the audit trail, retried if they fail, and undone in reverse order if a later step fails (for example, reversing a payout when fraud is

confirmed). Command turns each action into an object that CommandProcessor can execute, store, run again or undo, without needing to know what the action actually does.

Participants

GoF role	TIVIDY Class	Responsibility
Command	WorkflowCommand (abstract)	Declares execute() and undo(); stores who issued it, when, its status and attempt count
ConcreteCommand	AssignWorkItemCommand	Assigns a work item to a participant; undo restores the previous assignee.
ConcreteCommand	InvokeServiceCommand	Completes a work item with an outcome; undo reopens it
ConcreteCommand	DependencyTracker	Calls an external service and keeps its reference ID; undo asks the service to reverse it (e.g. refund).
Receiver	WorkItem, Participant, ExternalService	The objects that actually perform the work.
Invoker	CommandProcessor	Runs and retries commands, keeps them in its history, undoes them in reverse order, and reports each result to its WorkflowInstance through setState()
Client	WorkflowEngine	Creates concrete commands and binds them to their receivers

Collaboration

1. WorkflowEngine (Client) creates a concrete command, e.g. CompleteWorkItemCommand, and gives it its receiver (WorkItem, or ExternalService).
2. The engine passes the command to CommandProcessor (Invoker), which run() calls execute() through the abstract WorkflowCommand interface.

3. The concrete command calls its receiver to do the work, e.g. `item.complete()` or `service.call()`. Before it does, it saves whatever it needs to undo the action later.
4. The processor records each executed command in its history, so it can retry it later or call `undoAll()` on the commands in reverse order to reverse completed work

What part of the design is expected to vary

The part of the design expected to vary is the set of workflow actions and how each one is undone. New actions, such as approve, reject or cancel, can be added as new `WorkflowCommand` subclasses without changing `CommandProcessor`.

Why is Command appropriate?

Command encapsulates each request as an object. That is exactly what is needed to log it (audit), run it again (retry) and reverse it (compensation), while the processor stays independent of what each action does. Each `WorkflowInstance` owns its own `CommandProcessor`, because the commands belong to one run and are destroyed with it. This matches the definition-versus-instance split: the definition holds no commands, only the rules.

Task 6: Runtime behaviour diagrams

1. Workflow Sequence Diagram – Definition and instance creation

